

# GGTO · CANTV

Sistema de administración de reportes de avería y construcción de puntos ópticos

MANUAL LITERAL · DESPLIEGUE

## Manual de Cloud Run y Cloud SQL — GGTO (CANTV, Central Francisco Salias / Área 4)

Manual para todo público. Explica, en lenguaje sencillo, dónde vive la plataforma GGTO en Google Cloud: el servicio Cloud Run que atiende la aplicación, la base de datos Cloud SQL que guarda la información ggtov2, la imagen del contenedor, cómo se conecta todo y qué riesgos están vigentes. Cuando un dato no pudo comprobarse, se indica como «pendiente de confirmar». Este manual nunca escribe valores de secretos: solo los nombra.

---

### PRODUCTO

GGTO · versión 0.9.0

### COLECCIÓN

Manuales literales · Despliegue

### PÚBLICO

Todo público (lenguaje sencillo)

### CLIENTE

CANTV C.A. — Central Francisco Salias (Área 4)

### DOCUMENTO

docs/Manuales/04-Despliegue/02-cloud-run-y-cloud-sql.md

### ACTUALIZACIÓN DE LA FUENTE

28/09/2026

# ÍNDICE

|                                                                               |           |
|-------------------------------------------------------------------------------|-----------|
| <b>1 Empezar en 5 minutos</b>                                                 | <b>3</b>  |
| <b>2 Estado real del despliegue</b>                                           | <b>4</b>  |
| 2.1 Cloud Run ggto-web en truekeate-main/europe-west1                         | 5         |
| 2.2 URL y revision (ggto-web-00014-zqx, imagen v14)                           | 5         |
| 2.3 SA de ejecucion                                                           | 5         |
| <b>3 Cloud SQL</b>                                                            | <b>6</b>  |
| 3.1 instancia truekeate-db-dev (PostgreSQL 15.18)                             | 6         |
| 3.2 base ggtov2 y usuario ggtov2_app                                          | 6         |
| 3.3 socket /cloudsql/...                                                      | 7         |
| 3.4 SSL y roles endurecidos (entornos_globales.md:83-106)                     | 7         |
| <b>4 Imagen de contenedor</b>                                                 | <b>8</b>  |
| 4.1 multi-etapa Node+Python                                                   | 8         |
| 4.2 Artifact Registry truekeate-repo                                          | 8         |
| <b>5 Conexion de la aplicacion</b>                                            | <b>9</b>  |
| 5.1 socket vs proxy local 127.0.0.1:5433                                      | 9         |
| 5.2 app/core/db.py:26-36                                                      | 9         |
| <b>6 Despliegue a ggto-web</b>                                                | <b>10</b> |
| 6.1 Pasos operativos                                                          | 10        |
| 6.2 Alternativa por API REST                                                  | 11        |
| <b>7 Bloqueo de ggtov2 y plan de migracion (entornos_globales.md:360-369)</b> | <b>12</b> |
| <b>8 Riesgos</b>                                                              | <b>13</b> |
| 8.1 D-26 sin backups/PITR                                                     | 13        |
| 8.2 servicio publico con PII real (debe restringirse)                         | 13        |
| 8.3 roles de BD (NOINHERIT, riesgo residual)                                  | 13        |
| <b>9 Verificacion</b>                                                         | <b>14</b> |
| 9.0.1 Lista de comprobacion posterior al despliegue                           | 14        |

Manual para todo público. Explica, en lenguaje sencillo, **dónde vive la plataforma GGTO en Google Cloud**: el servicio Cloud Run que atiende la aplicación, la base de datos Cloud SQL que guarda la información `ggtov2`, la imagen del contenedor, cómo se conecta todo y qué riesgos están vigentes. Cuando un dato no pudo comprobarse, se indica como «pendiente de confirmar». Este manual **nunca escribe valores de secretos**: solo los nombra.

## 1 Empezar en 5 minutos

1. **Ubique el servicio.** GGTO corre en **Cloud Run** con el nombre `ggtov-web`, en el proyecto `truekeate-main` y la región `europa-west1`. Cloud Run es el servicio de Google que ejecuta la aplicación en contenedores y la publica por internet.
2. **Abra la dirección de la aplicación.** La dirección principal es `https://ggtov-web-593453426217.europa-west1.run.app`. La misma dirección sirve la pantalla (SPA en React) y la API: por eso `GET /`, los enlaces profundos del menú y todo `/api/v1/*` funcionan bajo la misma URL.
3. **Compruebe que está viva.** Agregue `/health` a la dirección. Debe responder `{"status": "ok"}`. Para saber si además alcanza la base de datos, use `/ready`.
4. **Sepa dónde están los datos.** La información vive en la base `ggtov2`, dentro de la instancia compartida `truekeate-main:southamerica-east1:truekeate-db-dev`, en PostgreSQL 15.18. El usuario de la aplicación es `ggtov2_app`.
5. **Recuerde las dos advertencias.** El servicio es **público** y ya contiene PII real (42 casos): debe restringirse. Además, la instancia **no tiene respaldos ni recuperación a un punto en el tiempo** (riesgo aceptado **D-26**): no cargue más datos reales hasta migrar.

## 2 Estado real del despliegue

El despliegue definitivo **no** vive en el proyecto previsto `ggtov2`. La facturación de `ggtov2` está **bloqueada** porque el cupo de cinco proyectos de la cuenta de facturación está agotado. Por esa razón, el servicio se aloja temporalmente en el proyecto `truekeate-main`, que sí tiene facturación activa, apuntando a la base `ggtov2` de la instancia compartida.

| ELEMENTO                   | VALOR VERIFICADO                                                            |
|----------------------------|-----------------------------------------------------------------------------|
| Proyecto actual            | <code>truekeate-main</code>                                                 |
| Servicio                   | Cloud Run <code>ggtov-web</code>                                            |
| Región                     | <code>europa-west1</code>                                                   |
| URL principal              | <code>https://ggtov-web-593453426217.europa-west1.run.app</code>            |
| URL alternativa            | <code>https://ggtov-web-m33mjctj4a-ew.a.run.app</code>                      |
| Acceso                     | Público ( <code>allUsers</code> con el rol <code>roles/run.invoker</code> ) |
| Proyecto previsto a futuro | <code>ggtov2</code> ( número de proyecto <code>905974355709</code> )        |

El despliegue corresponde a la decisión **D-43** del registro de decisiones: «servicio Cloud Run `ggtov-web` en `truekeate-main / europa-west1`, con cuenta de servicio propia `ggtov-web-sa`, conectado a `ggtov2` por el conector de Cloud SQL». Esa decisión incluye la advertencia explícita de que el acceso público es una prueba de humo (*smoke test*) que debe restringirse antes de producción.

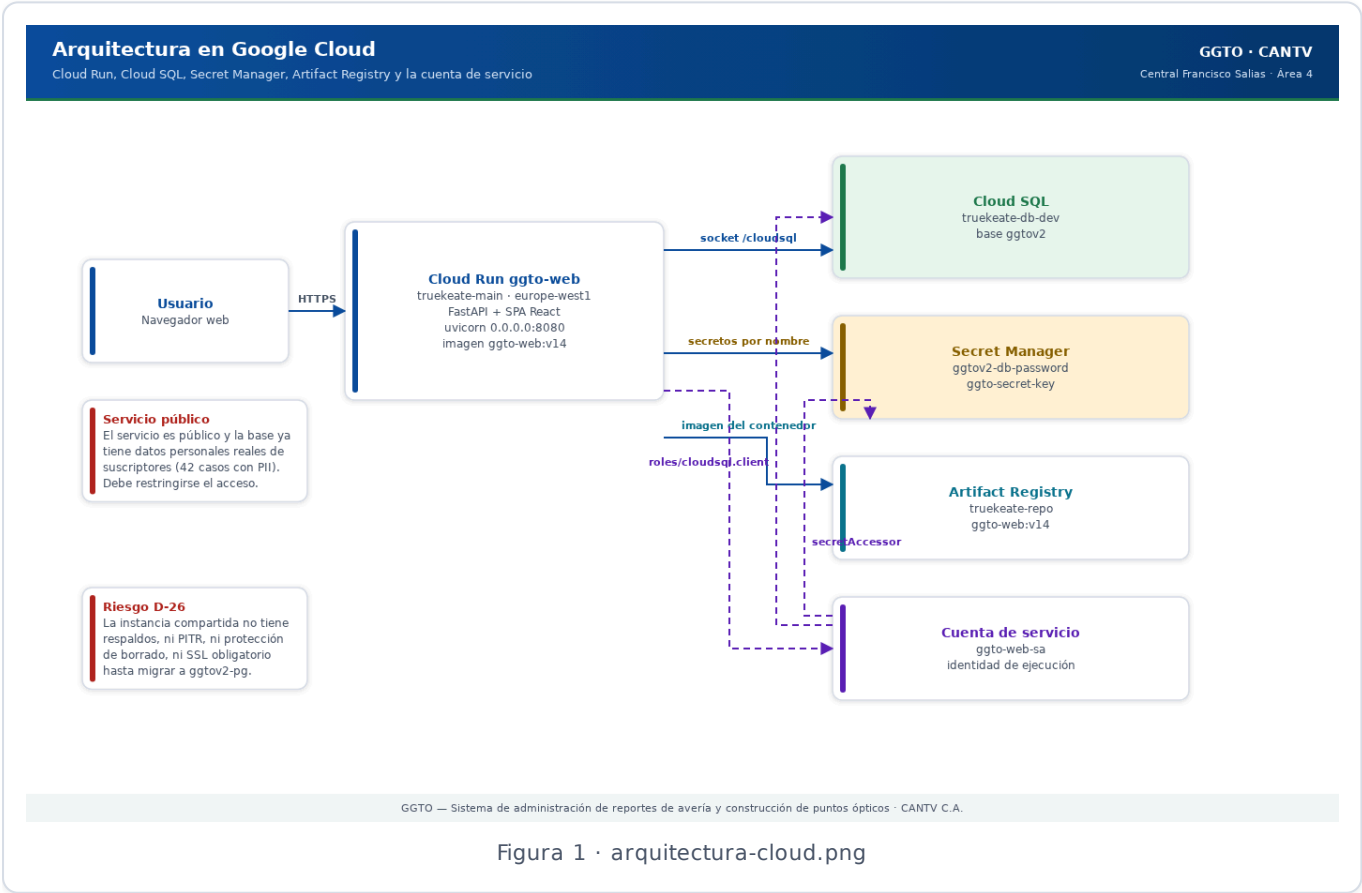


Figura 1 · arquitectura-cloud.png

## 2.1 Cloud Run ggto-web en truekeate-main/europe-west1

Cloud Run ejecuta el contenedor que contiene **dos cosas a la vez**: la API hecha en FastAPI y la pantalla o SPA (aplicación de una sola página) compilada en React. El punto de entrada del contenedor es `uvicorn app.main:app --host 0.0.0.0 --port 8080`. La aplicación monta la SPA al final del enrutador para no tapar la API: el componente `SPAStaticFiles` sirve la carpeta `app/web/dist` y responde `index.html` ante las rutas del cliente. Es decir, la misma URL sirve `GET /`, los enlaces profundos de React Router y todo `/api/v1/*`.

La configuración de ejecución verificada en el servicio incluye:

- Cuenta de servicio de ejecución `ggto-web-sa@truekeate-main.iam.gserviceaccount.com`, con los roles `roles/cloudsql.client` y `roles/secretmanager.secretAccessor` sobre los secretos `ggto-v2-db-password` y `ggto-secret-key`. Una cuenta de servicio es la «identidad» con la que el contenedor pide permisos a Google.
- La instancia Cloud SQL montada `truekeate-main:southamerica-east1:truekeate-db-dev`.
- El código desplegado es `app/` (FastAPI + SQLAlchemy + Argon2id + JWT) y `app/web/` (React + Vite + TypeScript).

## 2.2 URL y revision (ggto-web-00014-zqx, imagen v14)

La última revisión documentada del servicio es la siguiente. Una «revisión» es cada versión publicada del servicio; Cloud Run atiende una revisión a la vez.

| DATO                              | VALOR                                                                                     |
|-----------------------------------|-------------------------------------------------------------------------------------------|
| Revisión actual                   | <code>ggto-web-00014-zqx</code>                                                           |
| Imagen                            | <code>southamerica-east1-docker.pkg.dev/truekeate-main/truekeate-repo/ggto-web:v14</code> |
| Versión funcional                 | Ciclo 9 — ALERTAS, Telegram y MCP                                                         |
| Pruebas al cierre de esa revisión | 152/152 pruebas pytest                                                                    |
| Verificación en vivo              | 42 casos, 0 fallas, bandeja de salida vacía, 73 endpoints                                 |

La revisión inmediatamente anterior documentada fue `ggto-web-00013-9pf` con la imagen `v13`. Existe una secuencia completa de revisiones históricas por ciclo: `ggto-web-00002-rbl` (v2), `00003-6zb` (v3), `00004-cj9` (v4), `00005-rxm` (v5), `00006-htl` (v6), `00007-vsr` (v7), `00008-vr5` (v8), `00009-fvc` (v9), `00011-ksr` (v11), `00012-lbl` (v12), `00013-9pf` (v13) y `00014-zqx` (v14).

La revisión activa en el momento de leer este manual debe confirmarse con el comando `gcloud run services describe ggto-web --region=europe-west1 --project=truekeate-main`. El dato aquí consignado es el último documentado en el repositorio, no una lectura en vivo.

## 2.3 SA de ejecucion

La cuenta de servicio de ejecución es la identidad con la que el contenedor accede a Cloud SQL y a Secret Manager. Además de `ggto-web-sa@truekeate-main.iam.gserviceaccount.com`, el proyecto mantiene dos cuentas planificadas en el diseño original de `ggto-v2`:

| CUENTA                                                           | ROL PREVISTO                                               |
|------------------------------------------------------------------|------------------------------------------------------------|
| <code>ggtov2-app@ggtov2.iam.gserviceaccount.com</code>           | Cuenta de ejecución ( <i>runtime</i> ) del proyecto GGTOv2 |
| <code>ggtov2-deploy@ggtov2.iam.gserviceaccount.com</code>        | Cuenta de despliegue                                       |
| <code>ggtov-web-sa@truekeate-main.iam.gserviceaccount.com</code> | Cuenta de ejecución real del servicio                      |

La cuenta `ggtov2-app@ggtov2` conserva un permiso cruzado hacia `truekeate-main`: el rol `roles/cloudsql.client`. Ese permiso se otorgó consultando la política de permisos (`getIamPolicy`) y volviéndola a fijar (`setIamPolicy`) sobre `truekeate-main`, añadiendo el enlace con la cuenta de GGTOv2.

## 3 Cloud SQL

### 3.1 instancia truekeate-db-dev (PostgreSQL 15.18)

La base de datos operativa de GGTO es una base dedicada **dentro de una instancia compartida** con el proyecto TrueKeate. La separación es lógica (por base de datos), no física (por instancia). Este aspecto está señalado como riesgo en la documentación de infraestructura.

| DATO                                   | VALOR                                                                       |
|----------------------------------------|-----------------------------------------------------------------------------|
| Instancia                              | <code>truekeate-main:southamerica-east1:truekeate-db-dev</code>             |
| Motor                                  | <code>POSTGRES_15</code> / versión <code>15.18</code>                       |
| Región de la instancia                 | <code>southamerica-east1</code>                                             |
| Tipo y disco                           | <code>db-f1-micro</code> , 10 GB PD_SSD, ZONAL                              |
| IP pública                             | <code>34.39.180.101</code> (red autorizada <code>35.232.138.181/32</code> ) |
| SSL                                    | <code>ALLOW_UNENCRYPTED_AND_ENCRYPTED</code> (no obligatorio)               |
| Backups / PITR / protección de borrado | ✗ / ✗ / ✗                                                                   |

El esquema completo de la base (`RepoTecnico/db/schema.sql`, 38 821 bytes) se aplicó el 2026-09-26 en una sola transacción y sin errores. La verificación posterior confirmó 35 tablas, las extensiones `pgcrypto 1.3` y `pg_trgm 1.6`, seguridad por fila (RLS) en las tablas `caso` y `despacho`, 14 disparadores `actualizado_en`, 2 índices trigram y los datos iniciales previstos.

### 3.2 base ggtov2 y usuario ggtov2\_app

| RECURSO             | VALOR                                                                                 |
|---------------------|---------------------------------------------------------------------------------------|
| Base de datos       | <code>ggtov2</code> (juego de caracteres UTF8)                                        |
| Usuario             | <code>ggtov2_app</code> (contraseña aleatoria de 32 caracteres, host <code>%</code> ) |
| Contraseña          | Secret Manager <code>projects/truekeate-main/secrets/ggtov2-db-password</code>        |
| Secreto de conexión | <code>projects/truekeate-main/secrets/ggtov2-db-connection</code>                     |

La base y el usuario se crearon con el guion `10_usar_instancia_truekeate.sh`, que es idempotente: si la base ya existe no la vuelve a crear, y si el usuario ya existe **no le cambia la contraseña**. El guion genera la contraseña con `openssl rand -base64 36`, la filtra a caracteres alfanuméricos y la recorta a 32 caracteres sin imprimirla. En cada ejecución publica una versión nueva del secreto.

### 3.3 socket /cloudsql/...

La aplicación reconoce el socket Unix de Cloud SQL por una convención simple: si `DB_HOST` comienza con `/`, la dirección se construye sin puerto y el servidor viaja como parámetro de consulta `host=`, que es la forma que espera el conector. El valor que se monta en el contenedor es:

```
DB_HOST=/cloudsql/truekeate-main:southamerica-east1:truekeate-db-dev
DB_PORT=5432
DB_NAME=ggto2
DB_USER=ggto2_app
DB_SSLMODE=require
```

En Cloud Run la instancia se monta con la opción `--add-cloudsql-instances=truekeate-main:southamerica-east1:truekeate-db-dev`. La ventaja del socket es que el conector usa TLS (cifrado) y **no requiere abrir redes**, dado que la cuenta de servicio ya tiene `roles/cloudsql.client` en `truekeate-main`.

Detalle a verificar: el guion `07_cloudrun_web.sh` del repositorio **no** incluye `--add-cloudsql-instances` ni el `DB_HOST` del socket. Su cuerpo declara `DB_HOST=/cloudsql/truekeate-main:southamerica-east1:ggto2-pg`, apuntando a la instancia **planificada** de `ggto2` y no a la instancia compartida que realmente se usa. El montaje efectivo de la instancia compartida se hizo por fuera de ese guion; el comando exacto empleado está **pendiente de confirmar**.

### 3.4 SSL y roles endurecidos (entornos\_globales.md:83-106)

El endurecimiento del rol `ggto2_app` (es decir, quitarle permisos que no necesita) se aplicó el 2026-09-25 y está verificado:

| CAMBIO                                                                                                                                                                          | ESTADO                                                                                                                     |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| <code>CREATEROLE</code> revocado                                                                                                                                                | ✓                                                                                                                          |
| <code>CREATEDB</code> revocado                                                                                                                                                  | ✓                                                                                                                          |
| <code>NOINHERIT</code> aplicado (neutraliza <code>cloudsqlsuperuser</code> )                                                                                                    | ✓                                                                                                                          |
| Membresía <code>cloudsqlsuperuser</code> revocada                                                                                                                               | ⚠ No fue posible por SQL: Cloud SQL no otorga <code>ADMIN OPTION</code> ; la pertenencia persiste pero no se hereda        |
| <code>CONNECT</code> / <code>TEMPORARY</code> / <code>CREATE</code> de <code>PUBLIC</code> revocados en <code>truekeate</code> , <code>postgres</code> y <code>template1</code> | ✓ (el usuario <code>ggto2_app</code> recibe «permission denied for database» al intentar entrar a <code>truekeate</code> ) |
| <code>CONNECT</code> re-otorgado a <code>app</code> y <code>postgres</code>                                                                                                     | ✓ (TrueKeate sigue operando, con 39 tablas)                                                                                |
| <code>CONNECT</code> + <code>CREATE</code> en la base <code>ggto2</code> y el esquema <code>public</code>                                                                       | ✓ (el esquema de prueba se aplica correctamente)                                                                           |
| <code>truekeate-app-sa</code> sin acceso a los secretos de GGTO                                                                                                                 | ✓ (solo <code>ggto2-app@ggto2</code> figura en el IAM de ambos secretos)                                                   |

El estado original (antes del endurecimiento) estaba documentado con `createrole=True`, `createdb=True`, `cloudsqlsuperuser=True` y `CREATE_on_public=True`, lo que confirma que la corrección fue efectiva.

Sobre el SSL: la instancia permanece con `sslMode: ALLOW_UNENCRYPTED_AND_ENCRYPTED` porque cambiarla a `ENCRYPTED_ONLY` afecta a la aplicación TrueKeate, que comparte la instancia. La recomendación documentada es fijar `ENCRYPTED_ONLY`, dado que el conector de Cloud SQL siempre cifra y la aplicación coexistente no debería romperse. Sin embargo, **no se cambió sin autorización**, por tratarse de una instancia en uso.

## 4 Imagen de contenedor

### 4.1 multi-etapa Node+Python

La imagen se construye con el archivo `app/Dockerfile` y el contexto de construcción en el directorio `app/`. Es una construcción de dos etapas:

| ETAPA                            | BASE                          | ACCIONES                                                                            |
|----------------------------------|-------------------------------|-------------------------------------------------------------------------------------|
| 1 — <code>web</code>             | <code>node:20-slim</code>     | <code>npm ci</code> con <code>web/package*.json</code> y <code>npm run build</code> |
| 2 — ejecución ( <i>runtime</i> ) | <code>python:3.12-slim</code> | Instala <code>requirements.txt</code> y copia el código                             |

La segunda etapa fija `PYTHONDONTWRITEBYTECODE=1`, `PYTHONUNBUFFERED=1` y `PORT=8080`, instala las dependencias sin caché, copia el código a `/srv/app` y superpone el resultado de la etapa Node en `/srv/app/web/dist`. El comando final (`CMD`) arranca Uvicorn en el puerto 8080. El contexto de construcción usa el archivo `app/.dockerignore`.

Esta arquitectura multi-etapa es la que se verificó en el despliegue del Ciclo 2: «build multi-etapa Node+Python». El requisito **RNF-25** exige precisamente una «imagen Docker reproducible» y configuración solo por variables de entorno.

### 4.2 Artifact Registry truekeate-repo

Las imágenes se publican en el repositorio:

```
southamerica-east1-docker.pkg.dev/truekeate-main/truekeate-repo/ggto-web:v14
```

El nombre de la imagen sigue el patrón `<región>-docker.pkg.dev/<proyecto>/<repositorio>/<servicio>:<etiqueta>`. En el diseño original de `ggto2` el repositorio sería `europa-west1-docker.pkg.dev/ggto2/ggto2-web`, y el guion de despliegue usa por defecto `$REGION-docker.pkg.dev/$PROJECT_ID/ggto2-web/app:latest`. El repositorio realmente empleado es `truekeate-repo` en la región `southamerica-east1`, coherente con la ubicación de la instancia Cloud SQL.

El procedimiento de `docker build / docker push` y el proyecto de Cloud Build asociado para las imágenes `v1 ... v14` están **pendientes de confirmar**: la máquina de desarrollo no tiene `docker` instalado y no hay un archivo `cloudbuild.yaml` en el repositorio.

## 5 Conexión de la aplicación

### 5.1 socket vs proxy local 127.0.0.1:5433

GGTO se conecta a PostgreSQL de dos maneras según el entorno, y ambas convergen en la misma función `build_url()`:

| ENTORNO                | DB_HOST                                                      | DB_PORT | DB_SSLMODE |
|------------------------|--------------------------------------------------------------|---------|------------|
| Cloud Run (producción) | /cloudsql/truekeate-main:southamerica-east1:truekeate-db-dev | 5432    | require    |
| Desarrollo / E2E local | 127.0.0.1 (proxy)                                            | 5433    | disable    |

El proxy local se levanta con:

```
nohup /home/dsh/tools/cloud-sql-proxy \
  --project truekeate-main --address 127.0.0.1 --port 5433 \
  truekeate-main:southamerica-east1:truekeate-db-dev &
```

El puerto `5433` evita el choque con un PostgreSQL local en el puerto `5432`. La conectividad desde la máquina de desarrollo también admite conexión directa por la IP pública `34.39.180.101:5432` con SSL y la red autorizada `35.232.138.181/32`.

### 5.2 app/core/db.py:26-36

La lógica de conexión es la siguiente: si el servidor empieza por `/` se emite la dirección sin puerto y se añade el parámetro `host=`; en caso contrario se construye una dirección TCP con `host:puerto` y se añade `sslmode`; finalmente, si `DB_SCHEMA` tiene valor, se agrega `options=-csearch_path=<esquema>,public`.

El motor se crea con `pool_pre_ping=True`, `pool_size=5` y `max_overflow=5`. La opción `pool_pre_ping` hace que las conexiones muertas se detecten antes de usarse, algo importante en Cloud Run, donde la instancia puede reciclarse. La sesión por petición se entrega con `get_db()` y se cierra siempre.

`DB_PASSWORD` no aparece en el bloque de variables de Cloud Run de este manual por política de secretos. Se inyecta desde Secret Manager y su nombre (`ggto2-db-password`) sí está documentado.

## 6 Despliegue a ggto-web

El despliegue real se ejecuta con pasos equivalentes a los que describe el guion `07_cloudrun_web.sh`. Ese guion escribe el cuerpo JSON del servicio y lo crea o reemplaza mediante la API regional de Cloud Run, usando el ayudante `api()`, que obtiene un token de acceso por `$G00GLE_ACCESS_TOKEN`, por `gcloud`, por credenciales de usuario o por el servidor de metadatos.

Los elementos del despliegue que el guion fija son:

| ELEMENTO              | VALOR                                                                                         |
|-----------------------|-----------------------------------------------------------------------------------------------|
| Servicio              | <code>ggto2-web</code> (variable <code>SERVICE</code> )                                       |
| Imagen                | <code>\$IMAGE</code> o <code>\$REGION-docker.pkg.dev/\$PROJECT_ID/ggto2-web/app:latest</code> |
| Puerto del contenedor | <code>8080</code>                                                                             |
| Variables de entorno  | <code>DB_USER</code> , <code>DB_NAME</code> , <code>DB_HOST</code> , <code>SECRET_KEY</code>  |
| Recursos              | CPU <code>1</code> , memoria <code>512Mi</code>                                               |
| Escalado              | mínimo <code>0</code> , máximo <code>10</code>                                                |
| Tiempo de espera      | <code>300s</code>                                                                             |
| Acceso                | <code>roles/run.invoker</code> para <code>allUsers</code>                                     |

`SECRET_KEY` no se escribe en claro: se inyecta por referencia al secreto `ggto2-app-secret-key` con versión `latest`. El nombre del secreto efectivamente usado por el servicio desplegado es `ggto-secret-key`.

### 6.1 Pasos operativos

Los pasos siguientes reflejan el patrón del guion y la configuración verificada del servicio real. El servicio desplegado se llama `ggto-web` en `truekeate-main`; los marcadores entre ángulos deben sustituirse por valores reales sin escribir secretos en la línea de comandos.

## 1. Autenticarse y fijar el proyecto destino.

```
gcloud auth login
gcloud config set project truekeate-main
gcloud config set run/region europe-west1
```

El guion equivalente obtiene el token con el ayudante `gcp_token()`.

## 2. Construir y publicar la imagen en Artifact Registry (`southamerica-east1-docker.pkg.dev/truekeate-main/truekeate-repo/ggto-web:<etiqueta>`), siguiendo la construcción multi-etapa de `app/Dockerfile`.

## 3. Crear o actualizar el servicio montando Cloud SQL y declarando los secretos. La forma canónica en `gcloud` es:

```
gcloud run deploy ggto-web \
  --project truekeate-main \
  --region europe-west1 \
  --image southamerica-east1-docker.pkg.dev/truekeate-main/truekeate-repo/ggto-web:<tag> \
  --port 8080 \
  --cpu 1 --memory 512Mi \
  --min-instances 0 --max-instances 10 \
  --timeout 300 \
  --add-cloudsql-instances truekeate-main:southamerica-east1:truekeate-db-dev \
  --set-env-vars DB_HOST=/cloudsql/truekeate-main:southamerica-east1:truekeate-db-dev,DB_PORT=5432,DB_NAME=gg \
  --set-secrets SECRET_KEY=ggto-secret-key:latest,DB_PASSWORD=ggto2-db-password:latest \
  --service-account ggto-web-sa@truekeate-main.iam.gserviceaccount.com
```

Las variables y los recursos de este comando reproducen el guion del repositorio (puerto, CPU, memoria, escalado y tiempo de espera) y el montaje del socket documentado. La opción `--add-cloudsql-instances` con la instancia compartida es el paso que **no** está en el guion del repositorio y que debe ejecutarse de forma explícita (ver la observación de la sección *socket /cloudsql/...*).

## 4. Permitir la invocación. El guion publica el servicio para `allUsers`. Esto es una prueba de humo y **debe restringirse antes de producción**. La alternativa recomendada es IAP (Identity Aware Proxy, que exige iniciar sesión) o la invocación autenticada.

## 5. Verificar la revisión, la URL y los endpoints de salud (ver la sección *Verificacion*).

## 6.2 Alternativa por API REST

El guion del repositorio no usa `gcloud run deploy`, sino la API v2 directamente: construye el cuerpo JSON con `python3` y hace `POST` `https://run.googleapis.com/v2/projects/$PROJECT_ID/locations/$REGION/services?serviceId=$SERVICE`. El IAM se fija después con `services/$SERVICE:setIamPolicy`. Una ventaja de esta vía es que el cuerpo JSON admite el bloque `vpcAccess` con la red `ggto2-vpc` y la subred `ggto2-subnet`, pensado para la salida VPC directa del diseño original.

## 7 Bloqueo de ggtov2 y plan de migración (entornos\_globales.md:360-369)

El proyecto `ggtov2` no puede alojar todavía el servicio. El estado del bloqueo es:

| REQUISITO                                            | ESTADO                                                                                                                                           |
|------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| Facturación en <code>ggtov2</code>                   | ✗ Desactivada (cupo de 5 proyectos agotado)                                                                                                      |
| APIs de despliegue en <code>ggtov2</code>            | ✗ <code>run</code> , <code>artifactregistry</code> , <code>cloudbuild</code> y <code>secretmanager</code> no se pueden habilitar sin facturación |
| Cloud Run / Artifact Registry en <code>ggtov2</code> | ✗ No existen                                                                                                                                     |

El plan de migración, cuando se desbloquee la facturación, es:

1. Ejecutar el guion `07_cloudrun_web.sh` para crear el servicio en `ggtov2` con su propia imagen en `europa-west1-docker.pkg.dev/ggtov2/ggtov2-web`.
2. Crear y migrar a un Cloud SQL propio `ggtov2-pg` (PostgreSQL 16) con el tipo `db-custom-2-7680`, alta disponibilidad `REGIONAL`, SSL `ENCRYPTED_ONLY`, respaldos y PITR (recuperación a un punto en el tiempo).
3. Migrar los datos de `ggtov2` (instancia compartida) al Cloud SQL propio y reapuntar `DB_HOST` al nuevo socket.

La migración cierra el riesgo aceptado **D-26** y cumple **RNF-16** (respaldo con  $RPO \leq 24$  h y  $RTO \leq 4$  h) y **RNF-17** (disponibilidad  $\geq 99$  %). Hay un requisito previo de negocio: **liberar cupo o solicitar aumento de cuota** para habilitar `compute`, `run`, `secretmanager`, `cloudkms`, `storage`, `artifactregistry` y un Cloud SQL propio.

El plan de corte (ventana, herramienta de volcado y restauración, y validación de integridad) está **pendiente de confirmar**. La máquina de desarrollo no tiene `psql` ni `docker`, por lo que la ejecución del traslado requiere habilitar herramientas adicionales.

## 8 Riesgos

### 8.1 D-26 sin backups/PITR

El riesgo **D-26** está registrado y aceptado formalmente: no se modifican los respaldos, el PITR ni el SSL de `truekeate-db-dev`, por ser una instancia compartida de TrueKeate. El dueño es la Dirección del proyecto, la fecha de registro es 2026-09-25 y el cierre previsto es migrar a `ggtov2-pg` con respaldos, PITR y SSL al desbloquear la facturación. Incluye la instrucción explícita de **no cargar datos reales hasta entonces**.

El estado técnico que sustenta el riesgo es: respaldos ❌, PITR ❌, protección de borrado ❌ y SSL no obligatorio. Los requisitos **RNF-16** y **RNF-17** definen el objetivo a cumplir, y el documento de requerimientos advierte que la base provisional **no** cumple RNF-16 ni SSL.

**Consecuencia operativa:** ante una pérdida o corrupción de la base `ggtov2`, no existe restauración a un punto en el tiempo. Cualquier dato de producción cargado en esta instancia queda sin respaldo.

### 8.2 servicio publico con PII real (debe restringirse)

El servicio `ggtov-web` es accesible por `allUsers` (es decir, por cualquiera que tenga la dirección) y contiene **PII de suscriptores**: 42 casos reales cargados desde el archivo `detalle_averias_gpon_15_09_2026.csv` por el Super Usuario. La propia documentación advierte que «conviene restringir el acceso y evaluar el traslado a una instancia con respaldo (RNF-16) antes de seguir cargando información».

Este riesgo figura como pendiente externo de primer orden en el estado del proyecto: «Restringir el acceso público (ya hay PII real)». La mitigación recomendada es IAP o invocación autenticada. Adicionalmente, `CORS_ORIGINS` por defecto es `*`, lo que refuerza la necesidad de endurecer el acceso.

### 8.3 roles de BD (NOINHERIT, riesgo residual)

El rol `ggtov2_app` conserva la membresía `cloudsqlsuperuser` porque Cloud SQL no otorga `ADMIN OPTION` y no es posible revocarla por SQL. Con `NOINHERIT`, la pertenencia **no se hereda**, pero un `SET ROLE cloudsqlsuperuser` explícito podría recuperar privilegios. La corrección definitiva exige **recrear el rol**, algo inviable con los privilegios actuales: `DROP ROLE` exige ser miembro o superusuario, y el usuario `postgres` no es `rolsuper`. La decisión adoptada fue aceptar el riesgo residual con `NOINHERIT` y documentarlo.

El estado verificado del rol es: `CREATEROLE` revocado, `CREATEDB` revocado, `NOINHERIT` aplicado y sin acceso a las bases `truekeate`, `postgres` y `template1`.

## 9 Verificación

La verificación de un despliegue se apoya en tres endpoints de diagnóstico más el de metadatos. Todos están definidos en `app/api/routes_health.py`:

| ENDPOINT                         | TIPO                         | AUTENTICACIÓN  |
|----------------------------------|------------------------------|----------------|
| GET <code>/health</code>         | Liveness — no toca la base   | Público        |
| GET <code>/ready</code>          | Readiness — consulta la base | Público        |
| GET <code>/api/v1/info</code>    | Metadatos del servicio       | Público        |
| GET <code>/api/v1/resumen</code> | Conteos del esquema          | Requiere token |

`/health` devuelve `{"status":"ok"}` sin acceso a datos. `/ready` ejecuta una consulta que devuelve la versión de PostgreSQL, la base actual, el usuario y el número de tablas de `public`; si falla, responde **503** con el tipo de excepción. `/api/v1/info` devuelve `servicio`, `version`, `entorno` y la ruta de la documentación. `/api/v1/resumen` cuenta tablas, centrales, roles, cuadrillas, causas y parámetros, y también responde 503 si la base no está disponible.

Comandos de verificación posteriores a un despliegue:

```
# Revisión y URL del servicio
gcloud run services describe ggto-web \
  --project truekeate-main --region europe-west1 \
  --format='value(status.latestReadyRevisionName,status.url)'

# Salud (públicos)
curl -sf https://ggto-web-593453426217.europe-west1.run.app/health
curl -sf https://ggto-web-593453426217.europe-west1.run.app/ready
curl -sf https://ggto-web-593453426217.europe-west1.run.app/api/v1/info
```

Respuestas verificadas en producción al cierre del despliegue:

| ENDPOINT                         | RESPUESTA DOCUMENTADA                                                                                              |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------|
| GET <code>/</code>               | SPA React ( <code>index.html</code> + <code>/assets/*</code> ); enlaces profundos vía <code>SPASStaticFiles</code> |
| GET <code>/api/v1/info</code>    | <code>{"servicio":"GGTO API","version":"0.2.0","entorno":"production",...}</code>                                  |
| GET <code>/health</code>         | <code>{"status":"ok"}</code>                                                                                       |
| GET <code>/ready</code>          | <code>{"status":"ready","base":"ggto2","postgres":"PostgreSQL 15.18","tablas":35}</code>                           |
| GET <code>/api/v1/resumen</code> | □ requiere token                                                                                                   |

Discrepancia entre documentos: la respuesta de `/api/v1/info` consignada en el documento de entornos muestra `"version":"0.2.0"`, mientras que `APP_VERSION` por defecto en el código es `0.9.0` y el informe de Fase 4 evalúa `app/` como v0.9.0. El valor real en la revisión desplegada está **pendiente de confirmar**, leyendo el endpoint o la variable `APP_VERSION` del servicio.

Comprobaciones adicionales recomendadas tras cualquier despliegue:

### 9.0.1 Lista de comprobacion posterior al despliegue

1. **Registros sin errores de autenticación** contra Cloud SQL: la aplicación registra cada petición con `request_id`, método, ruta, código y duración mediante `ObservabilidadMiddleware`, y el formato de registro incluye marca de tiempo, nivel y nombre del registrador.
2. **Estado de los datos:** que `/ready` reporte `tablas: 35` y PostgreSQL `15.18`, coherentes con el esquema desplegado.
3. **Prueba de escritura:** un alta manual de caso debe generar un identificador `REF-<CENTRAL>-<NNNNNN>` mediante la función `generar_id_averia_ref`.
4. **Revisión de estado** con `gcloud run revisions list`, para confirmar que la nueva revisión recibe el 100 % del tráfico. Ese fue el procedimiento empleado al rotar la contraseña de TrueKeate.

Finalmente, el endpoint `ready` público no expone secretos, pero **sí revela** el nombre de la base, el usuario y la versión del motor. En un servicio con PII y acceso abierto, esa exposición es un argumento adicional para restringir el invocador y evaluar la restricción de `/ready` en producción; el endurecimiento concreto está **pendiente de confirmar**.